Non premtive

```
#include <stdio.h>

struct Process {
    int pid;
    int burst_time;
    int priority;
    int waiting_time;
    int turnaround_time;
    int completed;
};

int main() {
    int n, time = 0, completed = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct Process p[n];

    // Input
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter burst time and priority for process %d: ", p[i].pid);
        scanf("%d %d", &p[i].burst_time, &p[i].priority);
        p[i].completed = 0;
    }

    printf("\nExecution Order:\n");

    while (completed < n) {
        int idx = -1;
        int highest_priority = 9999;

        // Find process with highest priority (lowest number)
        for (int i = 0; i < n; i++) {
            if (!p[i].completed && p[i].priority < highest_priority) {
                highest_priority = p[i].priority;
                idx = i;
            }
        }

        // Execute selected process
        if (idx != -1) {
```

```
            p[idx].waiting_time = time;
            time += p[idx].burst_time;
            p[idx].turnaround_time = time;
            p[idx].completed = 1;
            completed++;

            printf("P%d ", p[idx].pid);
        }
    }

    // Display results
    printf("\n\nPID\tBT\tPriority\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t\t%d\t%d\n",
               p[i].pid,
               p[i].burst_time,
               p[i].priority,
               p[i].waiting_time,
               p[i].turnaround_time);
    }

    return 0;
}
```

premtive

```c
#include <stdio.h>

struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int remaining_time;
    int priority;
    int waiting_time;
    int turnaround_time;
};

int main() {
    int n;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct Process p[n];

    // Input
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter arrival time, burst time and priority for process %d: ", p[i].pid);
        scanf("%d %d %d", &p[i].arrival_time, &p[i].burst_time, &p[i].priority);
        p[i].remaining_time = p[i].burst_time;
    }

    int time = 0, completed = 0;

    printf("\nExecution Order:\n");

    while (completed < n) {
        int idx = -1;
        int highest_priority = 9999;

        // Find highest priority among arrived processes
        for (int i = 0; i < n; i++) {
            if (p[i].arrival_time <= time && p[i].remaining_time > 0) {
                if (p[i].priority < highest_priority) {
                    highest_priority = p[i].priority;
                    idx = i;
```

```
            }
        }
    }

    if (idx != -1) {
        // Execute for 1 unit (preemption possible)
        printf("P%d ", p[idx].pid);
        p[idx].remaining_time--;
        time++;

        // If process finishes
        if (p[idx].remaining_time == 0) {
            completed++;
            p[idx].turnaround_time = time - p[idx].arrival_time;
            p[idx].waiting_time = p[idx].turnaround_time - p[idx].burst_time;
        }
    } else {
        // CPU idle
        time++;
    }
}

// Output
printf("\n\nPID\tAT\tBT\tPriority\tWT\tTAT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t\t%d\t%d\n",
           p[i].pid,
           p[i].arrival_time,
           p[i].burst_time,
           p[i].priority,
           p[i].waiting_time,
           p[i].turnaround_time);
}

return 0;
}
```

reader-writer

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t wrt;      // controls writer access
sem_t mutex;    // protects readcount

int readcount = 0;

void *reader(void *arg) {
    int id = *((int *)arg);

    // Entry section
    sem_wait(&mutex);
    readcount++;
    if (readcount == 1) {
        sem_wait(&wrt); // first reader blocks writer
    }
    sem_post(&mutex);

    // Critical section
    printf("Reader %d is reading\n", id);
    sleep(1);

    // Exit section
    sem_wait(&mutex);
    readcount--;
    if (readcount == 0) {
        sem_post(&wrt); // last reader releases writer
    }
    sem_post(&mutex);

    return NULL;
}

void *writer(void *arg) {
    int id = *((int *)arg);

    sem_wait(&wrt); // request exclusive access
```

```
    // Critical section
    printf("Writer %d is writing\n", id);
    sleep(2);

    sem_post(&wrt); // release access

    return NULL;
}

int main() {
    pthread_t r[3], w[2];
    int id[5];

    sem_init(&wrt, 0, 1);
    sem_init(&mutex, 0, 1);

    // Create reader threads
    for (int i = 0; i < 3; i++) {
        id[i] = i + 1;
        pthread_create(&r[i], NULL, reader, &id[i]);
    }

    // Create writer threads
    for (int i = 0; i < 2; i++) {
        id[i + 3] = i + 1;
        pthread_create(&w[i], NULL, writer, &id[i + 3]);
    }

    // Join threads
    for (int i = 0; i < 3; i++) {
        pthread_join(r[i], NULL);
    }

    for (int i = 0; i < 2; i++) {
        pthread_join(w[i], NULL);
    }

    sem_destroy(&wrt);
    sem_destroy(&mutex);

    return 0;
}
```

Producer consuer

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full, mutex;

void *producer(void *arg) {
    int item;
    for (int i = 0; i < 10; i++) {
        item = i + 1;

        sem_wait(&empty);   // wait if buffer is full
        sem_wait(&mutex);   // enter critical section

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % SIZE;

        sem_post(&mutex);   // exit critical section
        sem_post(&full);    // increase filled slots

        sleep(1);
    }
    return NULL;
}

void *consumer(void *arg) {
    int item;
    for (int i = 0; i < 10; i++) {

        sem_wait(&full);    // wait if buffer is empty
        sem_wait(&mutex);   // enter critical section

        item = buffer[out];
```

```
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        sem_post(&mutex);   // exit critical section
        sem_post(&empty);   // increase empty slots

        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t p, c;

    sem_init(&empty, 0, SIZE); // initially all slots empty
    sem_init(&full, 0, 0);     // initially no items
    sem_init(&mutex, 0, 1);    // binary semaphore

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);

    return 0;
}
```

Dining philosopher

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_PHILOSOPHERS 5

// Mutexes represent the forks on the table
pthread_mutex_t forks[NUM_PHILOSOPHERS];

void* philosopher(void* num) {
    int id = *(int*)num;
    int left_fork = id;
    int right_fork = (id + 1) % NUM_PHILOSOPHERS;

    while (1) {
        // 1. Thinking
        printf("Philosopher %d is thinking.\n", id);
        sleep(1); // Simulate time spent thinking

        // 2. Pick up forks
        // Asymmetric condition to prevent deadlock
        if (id == NUM_PHILOSOPHERS - 1) {
            // Last philosopher picks up right fork first, then left
            pthread_mutex_lock(&forks[right_fork]);
            pthread_mutex_lock(&forks[left_fork]);
        } else {
            // All other philosophers pick up left fork first, then right
            pthread_mutex_lock(&forks[left_fork]);
            pthread_mutex_lock(&forks[right_fork]);
        }

        // 3. Eating
        printf("Philosopher %d is EATING.\n", id);
        sleep(2); // Simulate time spent eating

        // 4. Put down forks
        pthread_mutex_unlock(&forks[left_fork]);
        pthread_mutex_unlock(&forks[right_fork]);

        printf("Philosopher %d finished eating and put down forks.\n", id);
    }
```

```
    return NULL;
}

int main() {
    pthread_t threads[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];

    // Initialize the mutexes (forks)
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        if (pthread_mutex_init(&forks[i], NULL) != 0) {
            printf("Mutex initialization failed.\n");
            return 1;
        }
    }

    // Create the philosopher threads
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, philosopher, &ids[i]);
    }

    // Wait for threads to finish (this loop runs indefinitely in this setup)
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(threads[i], NULL);
    }

    // Destroy the mutexes (cleanup)
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}
```